

MÓDULO ESPECÍFICO PROFISSIONAL • 32h

Programador Front-End

Aula 07 — Git — Controle de Versão Profissional

FrontEnd Essencial

UC 1: Desenvolver interface web client-side

Carga horária: 32h • Aula 7 de 8

P R O F E S S O R

Alexandre Santana

ANALISTA DE SISTEMAS

R O T E I R O D E H O J E

Cronograma da Aula (4h)



1º bloco

50 min

O que é Git?

Controle de versão, repositório e commits



2º bloco

50 min

Fluxo de Trabalho

Working Area → Staging → Repository



3º bloco

50 min

Prática no Terminal

3 exercícios: config, commits e branches



4º bloco

45 min

GitHub

Publicar o projeto do Cartão de Visita

O que é Controle de Versão?

Definição Sistema que registra todas as mudanças feitas em arquivos ao longo do tempo, permitindo recuperar versões anteriores, trabalhar em equipe sem conflitos e entender o histórico completo do projeto.

✗ Sem controle de versão

```
index_final.html
index_final_v2.html
index_ESSE_MESMO.html
index_agora_vai.html
index_backup_27mai.html
```

↑ *Você conhece esse problema?*

✓ Com Git

```
index.html (apenas 1 arquivo!)
git log → histórico completo
git checkout → volta qualquer versão
branches → trabalho em paralelo
merge → integra sem perder nada
```

↑ *Isso é profissionalismo!*

C O N C E I T O F U N D A M E N T A L

O que é Git? Conceitos-chave

Git é o sistema de controle de versão distribuído mais usado no mundo (criado por Linus Torvalds em 2005). Cada desenvolvedor tem uma cópia **COMPLETA** do repositório na sua máquina — funciona mesmo offline.



Repositório

Pasta monitorada pelo Git. Contém todos os arquivos do projeto e o histórico completo de mudanças.



Commit

Fotografia salva do projeto em um momento. Cada commit tem uma mensagem descritiva e um código único (hash).



Branch

Ramificação paralela para desenvolver sem afetar o código principal (main). Permite trabalho simultâneo.



Merge

Une duas branches. Integra as mudanças desenvolvidas em paralelo sem perder nenhum trabalho.



Remote

Repositório remoto (ex.: GitHub). Servidor onde o código fica armazenado e compartilhado com a equipe.



Clone

Copia um repositório remoto para sua máquina local, trazendo todo o histórico e os arquivos.

C O N F I G U R A Ç Ã O I N I C I A L

Instalando e Configurando o Git

Instalação:

Windows

```
git-scm.com/download/win
```

macOS

```
brew install git
```

Verificar instalação: `git --version`

Linux/Ubuntu

```
sudo apt install git
```

```
bash - Configuração do Git

# Configuração obrigatória (1x só)
$ git config --global \
  user.name "Alexandre Santana"

$ git config --global \
  user.email "alexandre@senai.rj.br"

# Definir editor padrão (VS Code)
$ git config --global \
  core.editor "code --wait"

# Verificar configurações
$ git config --list
```

Essas configurações ficam salvas globalmente no seu sistema. Só precisa fazer uma vez por máquina.

Exercício 01 — Instalação e Configuração do Git



Configurando o Git pela primeira vez na sua máquina

- 1 Instale o Git em git-scm.com (se ainda não tiver).
- 2 Abra o terminal (Git Bash, PowerShell ou CMD).
- 3 Configure seu nome: `git config --global user.name`.
- 4 Configure seu e-mail: `git config --global user.email`.
- 5 Verifique: `git config --list` e `git --version`.

 10 min

 bash — Saída esperada

```
# Passo a passo completo
$ git --version
git version 2.44.0

$ git config --global \
  user.name "Seu Nome"

$ git config --global \
  user.email "email@senai.rj.br"

$ git config --list
user.name=Seu Nome
user.email=email@senai.rj.br
```

FLUXO DE TRABALHO

Working Area → Staging → Repository

O ciclo Git Todo arquivo passa por 3 áreas antes de ser salvo no histórico. Entender esse fluxo é a chave para usar Git corretamente.



Estados dos arquivos:

Untracked	Arquivo novo, ainda não monitorado pelo Git	Modified	Arquivo modificado na Working Area, não adicionado	Staged	Arquivo adicionado com <code>git add</code> , pronto para commit	Committed	Arquivo salvo no repositório local com <code>git commit</code>
------------------	---	-----------------	--	---------------	--	------------------	--

C O M A N D O S E S S E N C I A I S

Git na Linha de Comando

Comando	Descrição	Grupo
<code>git init</code>	Inicializa um repositório Git na pasta atual	Início
<code>git status</code>	Mostra o estado atual dos arquivos (modified, staged...)	Início
<code>git add .</code>	Adiciona TODOS os arquivos ao Staging Area	Staging
<code>git add arquivo.html</code>	Adiciona um arquivo específico ao Staging	Staging
<code>git commit -m "msg"</code>	Cria um commit com a mensagem descritiva entre aspas	Commit
<code>git log</code>	Exibe o histórico de commits com hash, autor e data	Histórico
<code>git log --oneline</code>	Histórico resumido (uma linha por commit)	Histórico
<code>git diff</code>	Mostra as diferenças entre a versão atual e o último commit	Histórico
<code>git checkout -- .</code>	DESFAZ alterações na Working Area (cuidado!)	Desfazer
<code>git reset HEAD arquivo</code>	Remove arquivo do Staging (sem perder as mudanças)	Desfazer

Exercício 02 — Primeiro Repositório e Commit

</> Criando seu primeiro repositório Git e fazendo o primeiro commit

- 1 Abra o terminal na pasta do seu projeto HTML.
- 2 Inicialize: git init
- 3 Verifique: git status (aparece tudo em vermelho).
- 4 Adicione tudo: git add .
- 5 Commit: git commit -m "feat: adiciona cartão de visita inicial"

 12 min

```

bash — Saída esperada

$ cd ~/Desktop/senai/frontend
$ git init
Initialized empty Git repository in .git/

$ git status
Untracked files: index.html, style.css

$ git add .
$ git status
Changes to be committed: index.html...

$ git commit -m \
"feat: adiciona cartão de visita"
1 file changed, 15 insertions(+)

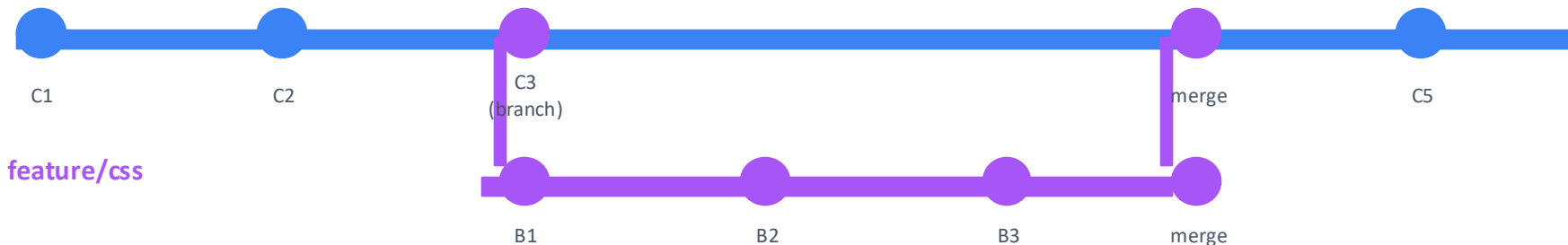
```

G I T A V A N Ç A D O

Branches — Trabalhando em Paralelo

Branch Uma linha de desenvolvimento independente. Você pode criar uma branch para cada nova funcionalidade (feature) sem afetar o código principal (main). Quando terminar, você faz o merge de volta.

main



Comandos de branches:

```
git branch feature/estilo
```

Cria uma nova branch

```
git checkout feature/estilo
```

Muda para a branch criada

```
git checkout -b feature/novo
```

Cria E muda de branch (atalho)

```
git merge feature/estilo
```

Une a branch à branch atual

```
git branch -d feature/estilo
```

Deleta a branch (após merge)

GitHub — Publicando seu Projeto

GitHub é a plataforma de hospedagem de repositórios Git mais popular do mundo (mais de 100 milhões de devs). Permite backup na nuvem, colaboração em equipe, portfólio público e integração com ferramentas de CI/CD.

Publicar projeto no GitHub:

- 1 Crie uma conta em github.com
- 2 Clique em "New repository"
- 3 Dê um nome (ex: cartao-visita-html)
- 4 NÃO inicialize com README (você já tem commits)
- 5 Copie a URL do repositório criado

```

# Conectar ao repositório remoto
$ git remote add origin \
  https://github.com/SEU-USER/REPO.git

# Enviar commits para o GitHub
$ git push -u origin main

Enumerating objects: 3, done.
Writing objects: 100% (3/3)

# Baixar atualizações do remoto
$ git pull origin main

# Clonar repositório existente
$ git clone https://github.com/...
  
```

git push = local → nuvem | git pull = nuvem → local | git clone = baixa o repositório inteiro

Exercício 03 — Publicando no GitHub



Publicando o repositório local no GitHub e vendo online!

- 1 Crie uma conta em github.com (se não tiver).
- 2 Crie um repositório: `cartao-visita-html` (público).
- 3 Copie a URL HTTPS do repositório.
- 4 No terminal: `git remote add origin URL`.
- 5 Envie: `git push -u origin main`

 10 min

```

bash – Publicação completa

# Verificar se já tem commits
$ git log --oneline
a1b2c3d feat: adiciona cartão de visita

# Conectar ao GitHub
$ git remote add origin \
  https://github.com/USUARIO/REPO.git

$ git push -u origin main
Enumerating objects: 3
To https://github.com/USUARIO/REPO.git
 * [new branch] main -> main

✓ Abra github.com/USUARIO/REPO

```

Boas Práticas e Commits Semânticos

Commits Semânticos (Conventional Commits):

feat

Nova funcionalidade

feat: adiciona formulário de contato

style

Apenas formatação/CSS

style: ajusta espaçamento do card

docs

Documentação

docs: adiciona README ao projeto

fix

Correção de bug

fix: corrige alinhamento do header

refactor

Refatoração sem nova feature

refactor: reorganiza funções JS

chore

Tarefas de configuração

chore: adiciona .gitignore

.gitignore — arquivos que o Git deve ignorar:

```
# .gitignore
node_modules/
.env
*.log
.DS_Store
```



Dicas de ouro:

- Commit cedo, commit sempre — saves frequentes
- Uma funcionalidade = um commit
- Nunca commit em main direto em equipe
- [gitignore.io](https://github.com/tj/gitignore.io) para gerar seu .gitignore

Quiz Rápido — 5 perguntas (5 min)

1. Qual comando INICIA um repositório Git?

- A) git start
- ✓ B) git init
- C) git create

2. O que git add . faz?

- A) Cria um commit
- ✓ B) Adiciona arquivos ao Staging
- C) Envia para o GitHub

3. Qual comando salva as mudanças no repositório LOCAL?

- A) git push
- B) git save
- ✓ C) git commit -m

4. git push envia commits para:

- A) O Staging Area
- ✓ B) O repositório remoto (GitHub)
- C) Uma nova branch

5. Qual tipo de commit semântico é para nova funcionalidade?

- A) fix
- B) chore
- ✓ C) feat

✓ = Resposta correta

Discuta cada resposta com o professor!

M ã O S N A M A S S A

Desafio Final — Versionar o Projeto Completo (20 min)



Versionando o Cartão de Visita HTML+CSS+JS com Git e publicando no GitHub!

Requisitos:

- ✓ Certifique-se que seu projeto tem: index.html, style.css e script.js
- ✓ git init (se ainda não inicializou)
- ✓ Crie um .gitignore e adicione node_modules/ e .DS_Store
- ✓ git add . → git commit -m "feat: projeto cartão de visita completo"
- ✓ Crie um repositório no GitHub (cartao-visita-frontend)
- ✓ git remote add origin URL → git push -u origin main



bash — Passo a passo completo

```
$ git init
$ echo 'node_modules/' >> .gitignore
$ git add .
$ git commit -m \
"feat: projeto cartão de visita"

$ git remote add origin \
https://github.com/USUARIO/REPO.git
$ git push -u origin main

🌐 Acesse: github.com/USUARIO/REPO
```

★ BÔNUS 1: Crie uma branch feature/dark-mode e adicione modo escuro | ★ BÔNUS 2: Ative o GitHub Pages (Settings → Pages) e veja seu site ao vivo!